Lab 7: Fuzzing for Input Validation Bugs

Software Engineering Lab

Name	Prathamesh Kumar Singh
SRN	PES1UG24CS341
Repository	L6_L7_33_spam_scoring (folder: lab7)
Application	`spam_scoring.py`
Test file	`test_spam_scoring.py` (Hypothesis fuzz tests)

1. Objective

Use property-based fuzz testing with the Hypothesis library to find input validation bugs in the three functions of `spam_scoring.py` (`sender_domain`, `first_link`, `shout_ratio`), and then fix the code so that it handles every unexpected or invalid input without crashing.

2. Bugs Found by Fuzzing

Function	Input that breaks it	Error raised
`sender_domain`	Text without '@' (e.g. hello)	IndexError
`sender_domain`	None	AttributeError
`first_link`	Text without 'http'	ValueError
`first_link`	None	AttributeError
`shout_ratio`	Text with no letters (e.g. 1234 or empty)	ZeroDivisionError
`shout_ratio`	None	TypeError

3. Fixes Applied

Function	Fix
`sender_domain`	Returns an empty string if the input is not a string or has no '@'; takes the part after the last '@', strips spaces and lower-cases it.
`first_link`	Returns an empty string if the input is not a string or has no 'http'; the link now ends at any whitespace (space, tab, newline).
`shout_ratio`	Returns 0.0 if the input is not a string or contains no letters, avoiding division by zero.
`main`	Prints a clear message for invalid input instead of crashing.

4. Test Cases Written

8 tests in `test_spam_scoring.py`: three no-crash fuzz tests (one per function, input `st.text() | st.none()`), four property tests that check the output is always valid (e.g. ratio is between 0 and 1, the link has no whitespace), and one test with known example inputs.

Step 2: Running the Buggy Application

Entering an email without '@' crashes `sender_domain` with an IndexError.

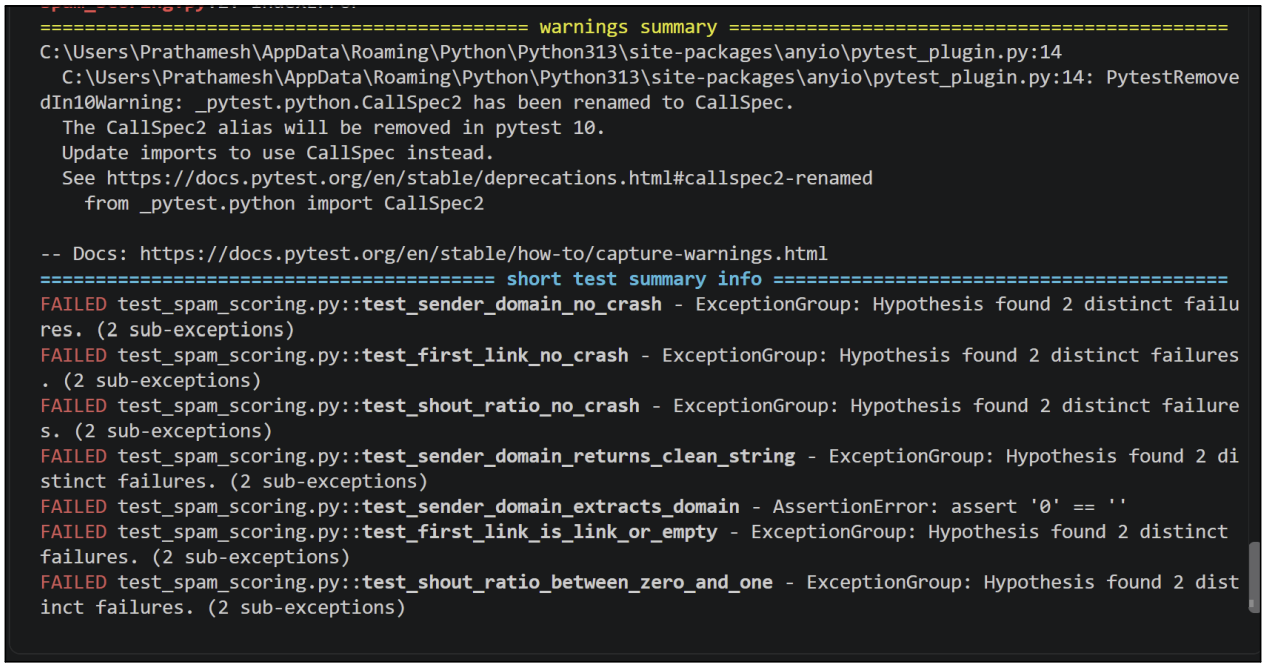
Command: `python spam_scoring.py`

```
PS D:\SE LAB\L6_L7_33_spam_scoring\lab7> python spam_scoring.py
==== Running sender_domain ====
Enter a sender email (e.g. offers@Deals.com): hello
Traceback (most recent call last):
  File "D:\SE LAB\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 33, in <module>
    main()
    ~~~~^^
  File "D:\SE LAB\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 19, in main
    result = sender_domain(user_input)
  File "D:\SE LAB\L6_L7_33_spam_scoring\lab7\spam_scoring.py", line 2, in sender_domain
    return email.split('@')[1].lower()
           ~~~~~~~~~~~~~~~~~~~~^^^
IndexError: list index out of range
```

Step 4: Hypothesis Tests Failing on the Buggy Code

The fuzz tests expose crashes in all three functions.

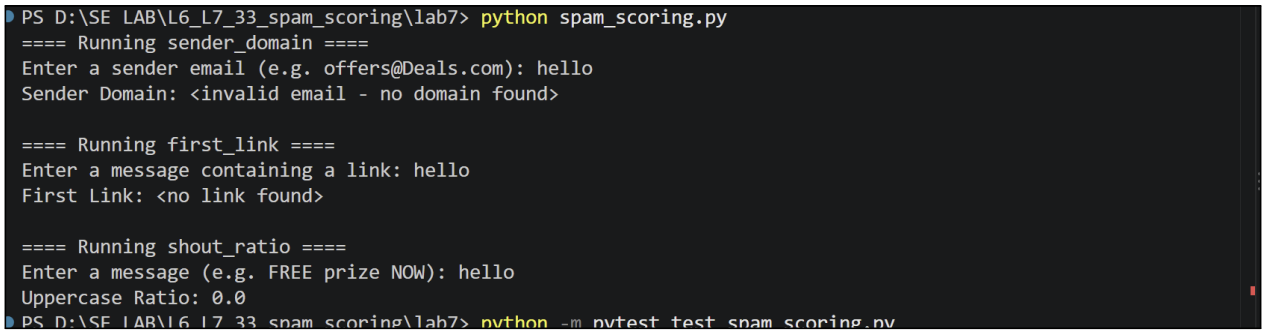
Command: `python -m pytest test_spam_scoring.py`



Step 5: Running the Fixed Application

Invalid inputs are now handled gracefully instead of crashing.

Command: `python spam_scoring.py`



Step 6: Hypothesis Tests Passing on the Fixed Code

All 8 fuzz tests pass.

Command: `python -m pytest test_spam_scoring.py`

```
PS D:\SE LAB\L6_L7_33_spam_scoring\lab7> python -m pytest test_spam_scoring.py
========================================= test session starts =========================================
platform win32 -- Python 3.13.13, pytest-9.2.0, pluggy-1.7.0
rootdir: D:\SE LAB\L6_L7_33_spam_scoring\lab7
plugins: anyio-4.13.0, hypothesis-6.170.15
collected 8 items

test_spam_scoring.py ........                                                          [100%]

========================================== warnings summary ===========================================
C:\Users\Prathamesh\AppData\Roaming\Python\Python313\site-packages\anyio\pytest_plugin.py:14
  C:\Users\Prathamesh\AppData\Roaming\Python\Python313\site-packages\anyio\pytest_plugin.py:14: PytestRemov
dIn10Warning: _pytest.python.CallSpec2 has been renamed to CallSpec.
  The CallSpec2 alias will be removed in pytest 10.
  Update imports to use CallSpec instead.
  See https://docs.pytest.org/en/stable/deprecations.html#callspec2-renamed
    from _pytest.python import CallSpec2

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
================================== 8 passed, 1 warning in 4.43s ===================================
```

5. Conclusion

Fuzz testing with Hypothesis found six crashes that the original example input would never have shown. After adding input validation and safe default return values, the application handles every generated input and all 8 tests pass.